

Why Did It Drop?

An analytics agent that explains why a product metric moved, and says when it can't. Tested on incidents planted in real event data, with every case published.

Akshay Dhar · akshaykumardhar14@gmail.com · github.com/AKSHAYKUMARDHAR/Why-Did-It-Drop

MY ROLE	WHEN	STACK
Solo: research, PRD, benchmark design, evaluation and the build, with Claude Code.	Oct 2026	Python, DuckDB, Gemini 3.5 Flash-Lite, GA4 event data, GitHub Pages
70 test cases: incidents planted in Google's real GA4 data, frozen before any prompt		3 real tracking failures found in Google's own demo data, used as test cases
26 = 26 development cases right, model versus fixed rules: the model added no decision value		Gate failed 20% wrong causes on the held-out run (gate: 5%), mostly the data's own campaign shifts

The problem

When a product metric moves, a PM asks "why?", and an analyst spends hours slicing dashboards. In 2026 Amplitude launched an agent that "investigates root causes, and explains what's driving changes across funnels", and Mixpanel's RCA agent ranks "which dimensions explain the shift". Neither publishes how often it's right.

The vendors' own material shows how these agents fail. In the winning story of Amplitude's 2026 AI Impact Awards, the agent first blamed the wrong release, until the user made it count error rates rather than raw errors. Mixpanel's guide warns that if events weren't tracked correctly, "RCA will confidently explain the wrong thing".

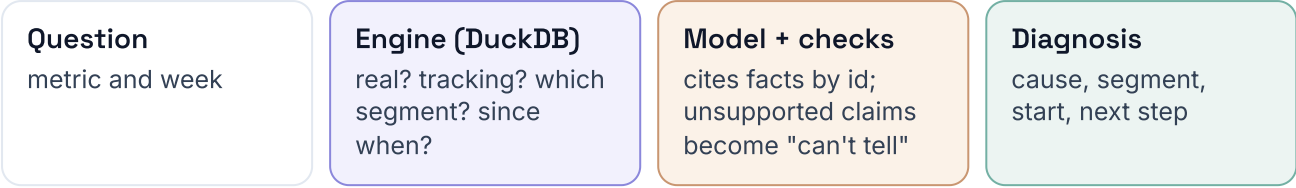
Who it's for

- **The PM at a consumer app** who needs to know, before standup, whether a drop is real, where it sits and what likely caused it.
- **The analyst on call** who wants the first hour of slicing done, every query shown, and an honest "the data can't explain this".

What I built

An investigator over e-commerce event data. Ask "what changed in conversion this week, and why?". Code checks whether the change is beyond normal weekly variation, then runs tracking-health checks for signatures no shopper produces: duplicate purchase events, purchases with no checkout, orders with zero revenue. It then finds the segment that behaved differently from the rest of the site and dates when it split. A

model reads that evidence and writes the diagnosis in plain English. Every number is a numbered fact the model cites by id, and code prints the value.



Why Did It Drop?

An analytics agent that explains why a product metric moved, and says when it can't. Every case below is a real week of Google's demo-store data. Most have an incident planted in them, with the answer known. You can see what the agent concluded, what was really there, and the evidence it saw.

28/40

add-to-cart rate right

20%

revenue (yearly, %)

17%

new items (last week, weekly %)

26/30

development cases right

100%

if current client case from the data takes the model typed half were removed

How it decides

1. Is the change real? This week against the usual week-to-week swing.

2. Can the data be trusted? Tracking checks look for signatures no shopper produces: duplicate purchase events, purchases with no checkout, orders with zero revenue.

3. Where is it? Segments that moved differently from the rest of the site, more than they usually do, in a way that fits a failure and made the asked metric worse.

4. What else it wasn't? A model (agent)-3.5-Rush-Reg reads that evidence and writes the diagnosis, citing every number by reference. Code IDs in the figures and checks each claim, anything unsupported becomes "can't tell".

Set

Hold-out release run (43)

Outcomes

Any

Case

Any

40 cases

100

Planted incident

Right

What changed in checkout start after add-to-cart in the week of 14 Dec - 20 Dec 2020, and why?

Conversion rate (sessions with a purchase) = Web, from 2020-12-18

Checkstart after add-to-cart declined due to a tracking issue affecting the Web operating system.

• Checkstart after add-to-cart dropped significantly while purchases without checkout increased for the Web operating system (starting from 11 (number removed) Dec (number removed)).

• This tracking finding directly affects the metric, and explains the observed drop.

• The issue is isolated to the Web operating system segment.

Next step: Investigate the analytics tag implementation and event firing logic for checkstart on the Web platform.

End of the model case

100

Quiet week

Right

What changed in revenue in the week of 14 Dec - 20 Dec 2020, and why?

Revenue = 17% week over week, within normal variation.

No real change

Revenue fell within normal weekly variation compared to the previous week.

• Revenue decreased from \$15,721 to \$47,276, which is within normal historical weekly variation (3.3).

• Both sessions 14% and average order value +10% remained stable, while conversion rate declined -10%.

• No tracking issues or candidate causes were found to explain the shift.

Next step: Review general traffic and seasonal trends to monitor future weekly performance.

End of the model case

103

Planted incident (type unseen in development)

Right

What changed in conversion rate (sessions with a purchase) in the week of 04 Jan - 10 Jan 2021, and why?

Conversion rate (sessions with a purchase) = 10% week over week, within normal variation.

Product break in Landing page section = Shop by Brand, from 2021-01-01

Conversion rate changes were driven by a sharp product-view rate drop on the Shop by Brand landing page.

• Although overall conversion variation remained within normal bounds, visitors landing on the Shop by Brand section experienced a dramatic product-view rate decline.

• This issue began on Tuesday and heavily impacted the funnel performance for that specific segment.

• Since this segment accounts for a portion of traffic, the localized breakdown changed down overall metrics.

Next step: Check recent deployments or content updates affecting the Shop by Brand landing page starting from Tuesday.

End of the model case

103

Planted incident (type unseen in development)

Right

What changed in conversion rate (sessions with a purchase) in the week of 04 Jan - 10 Jan 2021, and why?

Conversion rate (sessions with a purchase) = 10% week over week, within normal variation.

Product break in Landing page section = Shop by Brand, from 2021-01-01

Conversion rate changes were driven by a sharp product-view rate drop on the Shop by Brand landing page.

• Although overall conversion variation remained within normal bounds, visitors landing on the Shop by Brand section experienced a dramatic product-view rate decline.

• This issue began on Tuesday and heavily impacted the funnel performance for that specific segment.

• Since this segment accounts for a portion of traffic, the localized breakdown changed down overall metrics.

Next step: Check recent deployments or content updates affecting the Shop by Brand landing page starting from Tuesday.

End of the model case

103

Planted incident (type unseen in development)

Wrong cause

What changed in revenue in the week of 07 Dec - 13 Dec 2020, and why?

Revenue = 150% week over week, beyond normal variation.

Tracking issue in Browser = Android Webview, from 2020-12-11

Revenue surged beyond normal variation due to a tracking issue causing an abnormal jump in average order value.

• Revenue increased drastically, driven by a massive spike in average order value rather than normal traffic growth.

• Tracking health analysis identified a revenue per order jump starting on 11 (number removed) Dec (number removed).

• The data corruption effects overall revenue measurements and requires immediate technical review.

Next step: Investigate the checkout and payment logging for Android Webview to locate the source of the inflated order value.

End of the model case

103

Planted incident (type unseen in development)

Wrong cause

What changed in revenue in the week of 07 Dec - 13 Dec 2020, and why?

Revenue = 150% week over week, beyond normal variation.

Tracking issue in Browser = Android Webview, from 2020-12-11

Revenue surged beyond normal variation due to a tracking issue causing an abnormal jump in average order value.

• Revenue increased drastically, driven by a massive spike in average order value rather than normal traffic growth.

• Tracking health analysis identified a revenue per order jump starting on 11 (number removed) Dec (number removed).

• The data corruption effects overall revenue measurements and requires immediate technical review.

Next step: Investigate the checkout and payment logging for Android Webview to locate the source of the inflated order value.

End of the model case

The demo is a public benchmark explorer, not a chat box. Every one of the 70 cases is shown with the agent's diagnosis, what was really planted, the expected answer and the full evidence pack, and you can filter to see only the failures.

Product decisions

- **Tested, not trusted.** No one publishes accuracy for these agents. The product is the measurement as much as the agent.
- **Code computes, what the model explains.** The model never does arithmetic or writes SQL. It asks for metrics from a fixed definition layer and cites facts by reference.
- **Tracking first.** Health checks run before any search for a cause, because broken tracking is the failure vendors themselves warn about.
- **Rates, mix and noise before blame.** Each segment is tested against the rest of the site. A holiday that lowers every segment alike isn't an incident.
- **"Can't tell" is a valid answer.** A confident wrong cause sends engineers after the wrong release; the gate is built around it.

How I tested it

The data is Google's public GA4 export for its merchandise store: 4,295,584 events, checked against the count its uploader published, built into 360,129 sessions. Surveying it before building anything turned up three real tracking failures in Google's own demo data:

- add-to-cart events missing until 25 November;

- purchase events double-firing from 19 to 24 November, which inflated revenue;
- orders recorded with zero revenue from 26 January.

They became test cases, alongside a strong holiday season and weekly campaign shifts.

A seeded generator plants incidents into copies of the real sessions: checkout breaks on one OS, lost traffic from one source, bot bursts, duplicate or missing tracking events, basket drops. It places each only where no natural effect touches the metric. A difference-in-differences test labels each one "clear" or "too small to see" before any agent runs, and segments are scored by the sessions they actually cover, not their labels.

That gives 70 cases, frozen in git before any prompt existed: 30 for development and 40 held out, including two incident types development never saw. Development took five engine revisions, each a general rule written after reading the failures:

Development, rules only (30 cases)	Right	Wrong cause	False alarms
First engine	16	8	2 / 9
+ a segment must diverge more than it usually does	12	12	6 / 9
+ it must worsen in absolute terms	14	10	6 / 9
+ it must make the asked metric worse; rank by impact x severity	24	2	4 / 9
+ it must be on the asked metric's path · frozen	26	2	2 / 9
Held-out release run (40 cases), run once	Right	Wrong cause	False alarms
Rules only (no model)	30	7	2 / 12
Model only	27	9	2 / 12
Model + code checks · what ships	28	8	2 / 12

Release gate

- ✗ Wrong causes $\leq 5\%$: 8 of 40 (20%)
- ✗ Right on incidents $\geq 75\%$: 64%
- ✗ False alarms $\leq 10\%$ of quiet and holiday weeks: 2 of 12 (17%)
- ✓ Every number shown comes from a query: all of them

What failed, and what's next

The hardest competition for a planted incident was the real world. In early December a genuine surge of new visitors, who convert less, changed the funnel; over New Year, product views from the Home page really did drop. Both are true changes, and in 7 of the 12 held-out misses the agent named one of them instead of the planted incident. The benchmark placed each incident where nothing natural moved the same metric and dimension, but the agent searches every dimension.

An incident type the agent had never seen exposed a rule of mine. For a currency-units bug in one country, "when a tracking finding appears in overlapping segments, name the broadest" picked the operating system or browser carrying those orders. The rule was right for duplicate events; for a value jump, the right segment is where the signature is most concentrated.

The model added no decision value. With my first prompt it was worse than the fixed rules (18 against 26 of 30): it said "no real change" whenever the headline was within normal variation, even with a cause listed. With an explicit decision order it matched the rules case for case, and on held-out data the rules alone did slightly better. Its value is the explanation: a plain-English diagnosis and next step built from traced facts. Even there it typed dates itself in 24 of 69 answers; code removed every one. My first write-up claimed it typed none, because my summary read the wrong field. I found that through a screenshot and logged the correction.

Next: concentration rather than breadth for value-jump findings; ties broken by name, so cached answers always replay; and a "known campaign" input, the way Mixpanel asks teams for business context. Then a fresh held-out set, since fixes tuned on this one must be tested on cases they haven't seen. After that, the same benchmark run against vendor agents.

What I learned

- Real data fights back. Planted incidents are easy to find in synthetic data; in real data, true campaign shifts compete with them, and an agent that names a real change isn't simply wrong. A benchmark has to say what counts as the answer.
- Measure the model, don't assume it. Once the evidence is computed well, rules made the same calls; the model's job became explanation.
- Statistically real isn't the same as mattering. With tens of thousands of sessions, every small shift is "significant"; the rules that worked asked whether a segment diverged more than it usually does, got worse in absolute terms, and moved the metric asked about.
- Check your own reporting. The false "zero typed numbers" claim came from my code, not the model.

The data is Google's public, obfuscated GA4 demo export. The benchmark, generator and every model answer are in the repo, and every number comes from a logged run.